

Probe System Plan Critique

Summary: Overall Assessment: This plan has CRITICAL security vulnerabilities that must be addressed before implementation.

Generated: 2026-01-14 19:12 UTC | Ideas Extracted: 82

What Happened

Overall Assessment: This plan has CRITICAL security vulnerabilities that must be addressed before implementation. The Probe system will handle file operations, state capture, and data collection without proper security measures defined. Multiple unexamined assumptions about integration and behavior could cause catastrophic failures. Significant architectural complexity adds unnecessary risk.

Evidence: Existing Being system uses `0o600` for files, `0o700` for directories (see `src/waft/being.py:1964`) Scientific method tool's `StateCapture` doesn't set permissions (see `scientific_method_tool/state_capture.py:68`) Plan doesn't specify security measures for Probe storage.

Evidence: Existing Being system has `_validate_being_id()` method (see `src/waft/being.py:1949`) Plan doesn't mention validation for Probe IDs Scientific method tool doesn't validate experiment IDs.

Fix Required: Validate scientific method tool is available Test integration before use Handle tool failures gracefully Add fallback if tool unavailable Version pin scientific method tool.

Evidence Needed: Check Reality system API for observation methods Verify Being system exposes observable state Test observation capabilities.

Recommendation: Design analysis algorithm before implementation.

CRITICAL Security Vulnerabilities: 4 HIGH Safety Issues: 3 MEDIUM Unexamined Assumptions: 12
LOW Overengineering: 5 Oversights: 8 Missed Obviousness: 6.

Recommendation: DO NOT PROCEED until all CRITICAL and HIGH priority issues are addressed.
The security vulnerabilities alone make this plan unsafe to implement as-is.

This plan has CRITICAL security vulnerabilities that must be addressed before any code is written. The Probe system will handle file operations, state capture, and data collection without proper security measures. These are not minor issues - they are show-stoppers.

Recommendation: DO NOT PROCEED with implementation until all CRITICAL and HIGH priority issues are addressed. The security vulnerabilities alone make this plan unsafe to implement as-is.

Date: 2026-01-14 09:55:39 PST Plan: OriginPoint Probe Experimental System Critique Mode: Bad Faith / Adversarial Security-First Analysis: YES.

Fix Required: Set restrictive file permissions: `0o600` for files, `0o700` for directories Apply permissions after file creation in all Probe storage operations Validate storage path is within project (path traversal protection) Never store sensitive data (sanitize before storage).

Impact: Path traversal attacks (read/write files outside project) DoS attacks (resource exhaustion from long paths) File system corruption (null bytes in filenames) Log injection (newlines in IDs).

Fix Required: Validate all IDs before use in file paths Reject IDs with `..`, `/`, `\`, null bytes, control characters Limit ID length (e.g., max 255 characters) Sanitize IDs (alphanumeric + underscore + hyphen only) Validate path encoding (UTF-8, handle encoding errors) Use `Path.resolve()` and check it's within project root.

```
def validate_probe_id(self, probe_id: str) -> bool: """Validate probe_id is safe for file system use.""" if not probe_id or len(probe_id) > 255: return False if any(char in probe_id for char in ['.', '/', '\\', '\\x00']): return False if not probe_id.replace(".", "").replace('-', "").isalnum(): return False return True.
```

Issue: Plan uses `scientific_method_tool/state_capture.py` which captures Being state including skills, fitness, personality.

Attack Vector: State capture reads Being objects without sanitization Sensitive Being data (personality, memories) stored in experiment files State snapshots could contain sensitive information No access control on who can capture state.

Evidence: `StateCapture.capture_being_state()` captures full Being state (see `scientific_method_tool/state_capture.py:99`) No sanitization of sensitive fields Plan doesn't specify what data should be excluded.

Fix Required: Sanitize Being state before capture (exclude sensitive fields) Define whitelist of safe fields to capture Never capture memories, lessons, or personal data Add access control for state capture operations Encrypt sensitive state data if storage is shared.

Attack Vector: Probe could access Reality it shouldn't have access to No validation that Probe belongs to Reality Cross-Reality data leakage.

Evidence: Plan doesn't specify Reality access control Existing Reality system may not have access control Probe could observe Beings in other Realities.

Fix Required: Validate Probe has access to Reality before operations Check `probe.reality_id` matches `reality.reality_id` Add Reality access control checks Log unauthorized access attempts.

Fix Required: Add try/except blocks for all file I/O Handle `IOError` , `PermissionError` , `OSError` Validate file operations succeed Implement retry logic for transient failures Log errors for debugging.

Fix Required: Limit observation_history size (e.g., last 1000 observations) Limit active hypotheses (e.g., max 10) Limit experiment records (e.g., archive old experiments) Add disk space checks before writes Implement data rotation/archival.

Issue: Reality system may not expose observation API Other Beings may not be observable Environmental features may not exist.

Evidence Needed: Test scientific method tool with Probe data Verify tool supports required features Check tool limitations.

Evidence Needed: Test D&D stat integration with personality system Verify enhancement vs. override behavior Check stat-to-personality mapping.

Issue: Interface may not work as expected Communication protocol undefined Error handling for failed suggestions unclear.

Evidence Needed: Design communication protocol Test suggestion/review cycle Handle edge cases (Probe suggests invalid action).

Issue: Phase transitions may not work Pattern recognition may fail Adaptive phase may not combine correctly.

Evidence Needed: Test phase transition logic Verify pattern recognition works Test adaptive exploration.

Issue: Analysis algorithm undefined Cause-effect detection may fail Reflection may not produce useful insights.

Evidence Needed: Define analysis algorithm Test cause-effect detection Verify reflection produces insights.

Disk Full: What happens if disk fills up during experiment? (No handling) Network Down: What if external dependencies unavailable? (No fallback) Process Killed: What if process killed mid-experiment? (No cleanup) System Under Load: What if system is under heavy load? (No throttling).

Path Traversal: Probe IDs with `../` could escape project directory Resource Exhaustion: No limits on observations/experiments Data Injection: Malicious data in observations could corrupt system State Manipulation: Probe could manipulate its own state maliciously.

Empty Reality: What if Reality has no Beings? (No handling) Invalid Observations: What if observations are malformed? (No validation) Concurrent Probes: What if multiple Probes run simultaneously? (Race conditions) Corrupted State: What if Probe state is corrupted? (No recovery).

Add Error Handling: Handle all file I/O errors, network errors Add Resource Limits: Limit observation history, experiments, data size Validate Scientific Method Integration: Test integration before use Add Action Validation: Validate Probe-suggested actions before execution.

Simplify Architecture: Consider if separate system is necessary Add Versioning: Version Probe system and data Add Documentation: API docs, usage guide, examples.

Additionally, there are multiple unexamined assumptions that could cause catastrophic failures, significant overengineering that adds unnecessary risk, and obvious oversights that should have been caught.

This critique assumes the worst and looks for all the ways things could fail. Address these issues before implementation.

Impact: Crashes on file system errors, data loss, corrupted state.

Additional Details

Issue: Plan assumes `scientific_method_tool/` works correctly without validation.

Recommendation: Validate Reality observation API before implementation.

Assumption: Scientific method tool can handle Probe-specific data structures.

Recommendation: Test integration with sample Probe data before full implementation.

Recommendation: Design and test piloting interface before full implementation.

Assumption: Exploration evolves from random → systematic → adaptive correctly.

Recommendation: Implement and test each phase separately before combining.

Assumption: External/Internal pressure loops work as designed.

Assumption: Probe updates behavior based on experiment results.

Issue: Learning algorithm undefined Update mechanism unclear Learning may not converge.

Evidence Needed: Define learning algorithm Specify update mechanism Test learning convergence.

Recommendation: Design learning algorithm before implementation.

Assumption: Observation system can observe Reality, Beings, environment.

Recommendation: Validate observation capabilities before implementation.

Assumption: Reflection system can analyze feedback loops and cause-effect.

| Recommendation: Validate spawning process before implementation.

Issue: Plan creates separate ProbeBeing class when Being subclass would work.

| Consideration: Could use Being subclass with Probe capabilities instead of separate system.

Issue: Plan has 8 separate components for experimental system.

Issue: D&D stats may not add value beyond existing personality system.

Issue: Separate piloting interface may be unnecessary for experimental system.

| Consideration: Could use simpler direct control for experimental system.

Fix Required: Add unit tests, integration tests, security tests.

| Fix Required: Add error recovery, state restoration, retry logic.

Fix Required: Add performance limits, optimization, monitoring.

Issue: Plan mentions documentation but doesn't specify what or how.

Fix Required: Specify documentation requirements (API docs, usage guide, examples).

Issue: Plan doesn't specify how to migrate Probe data if system changes.

Fix Required: Add migration strategy, versioning, backward compatibility.

Issue: Plan doesn't specify how to clean up old experiments/observations.

Fix Required: Add cleanup strategy, archival, data rotation.

Issue: Plan doesn't specify how to rollback if Probe system fails.

Fix Required: Add rollback strategy, state backup, recovery procedures.

Issue: Plan doesn't specify who can create/control Probes.

Fix Required: Add authentication, authorization, access control.

Fix Required: Add rate limiting, operation throttling.

Fix Required: Add size limits, validation, truncation.

Fix Required: Add action validation, whitelist of allowed actions.

Fix Required: Add Probe isolation, separate storage, access control.

Issue: Plan doesn't specify versioning for Probe system or data.

Fix Required: Add versioning, migration, backward compatibility.